

In [44]:

```
import os

import numpy as np

import pandas as pd

import seaborn as sns

import matplotlib.pyplot as plt

from wordcloud import WordCloud, STOPWORDS

import nltk
from nltk.corpus import stopwords
from nltk.stem.porter import PorterStemmer
from nltk.stem import WordNetLemmatizer
from nltk.tokenize import word_tokenize, sent_tokenize

from bs4 import BeautifulSoup

import re, string, unicodedata

from sklearn.metrics import classification_report, confusion_matrix, accuracy_score
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelBinarizer
from sklearn.preprocessing import LabelEncoder
from sklearn.model_selection import train_test_split
from sklearn.metrics import confusion_matrix

from string import punctuation

from nltk import pos_tag
from nltk.corpus import wordnet

import tensorflow as tf
from tensorflow import keras
from tensorflow.keras.layers import Dense, Embedding, LSTM, Dropout, Bidirectional
from tensorflow.keras.callbacks import ReduceLROnPlateau
from tensorflow.keras.models import Sequential
from tensorflow.keras.preprocessing import text, sequence
from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.preprocessing.sequence import pad_sequences
from tensorflow.keras import backend as K
from keras.engine.topology import Layer
from tensorflow.keras import initializers, regularizers, constraints
from keras.layers import Dense, Input, LSTM, Bidirectional, Activation, Conv1D
from keras.layers import Dropout, Embedding, GlobalMaxPooling1D, MaxPooling1D
from keras.layers import GlobalAveragePooling1D, BatchNormalization, concatenate
from keras.layers import Reshape, merge, Concatenate, Lambda, Average
from tensorflow.keras.models import Sequential, Model, load_model
from keras.callbacks import ModelCheckpoint
from tensorflow.keras.initializers import Constant
from keras.layers.merge import add
from keras.preprocessing import sequence
from keras.preprocessing.text import Tokenizer, text_to_word_sequence
from keras.utils import np_utils
```

In [45]:

```
dataset = pd.read_json('/Users/moe/Desktop/HuffPost_News_Dataset_Categories/N
dataset.drop(['authors', 'link', 'date'], axis = 1, inplace = True)
dataset.head()
```

Out[45]:

	category	headline	short_description
0	CRIME	There Were 2 Mass Shootings In Texas Last Week...	She left her husband. He killed their children...
1	ENTERTAINMENT	Will Smith Joins Diplo And Nicky Jam For The 2...	Of course it has a song.
2	ENTERTAINMENT	Hugh Grant Marries For The First Time At Age 57	The actor and his longtime girlfriend Anna Ebe...
3	ENTERTAINMENT	Jim Carrey Blasts 'Castrato' Adam Schiff And D...	The actor gives Dems an ass-kicking for not fi...
4	ENTERTAINMENT	Julianna Margulies Uses Donald Trump Poop Bags...	The "Dietland" actress said using the bags is ...

In [46]:

```
dataset.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 200853 entries, 0 to 200852
Data columns (total 3 columns):
#   Column                Non-Null Count  Dtype
---  -
0   category              200853 non-null object
1   headline              200853 non-null object
2   short_description     200853 non-null object
dtypes: object(3)
memory usage: 4.6+ MB
```

In [47]:

```
dataset.describe()
```

Out[47]:

	category	headline	short_description
count	200853	200853	200853
unique	41	199344	178353
top	POLITICS	Sunday Roundup	
freq	32739	90	19712

In [48]:

```
print("We have a total of {} categories".format(dataset['category'].nunique())
dataset['category'].value_counts())
```

We have a total of 41 categories

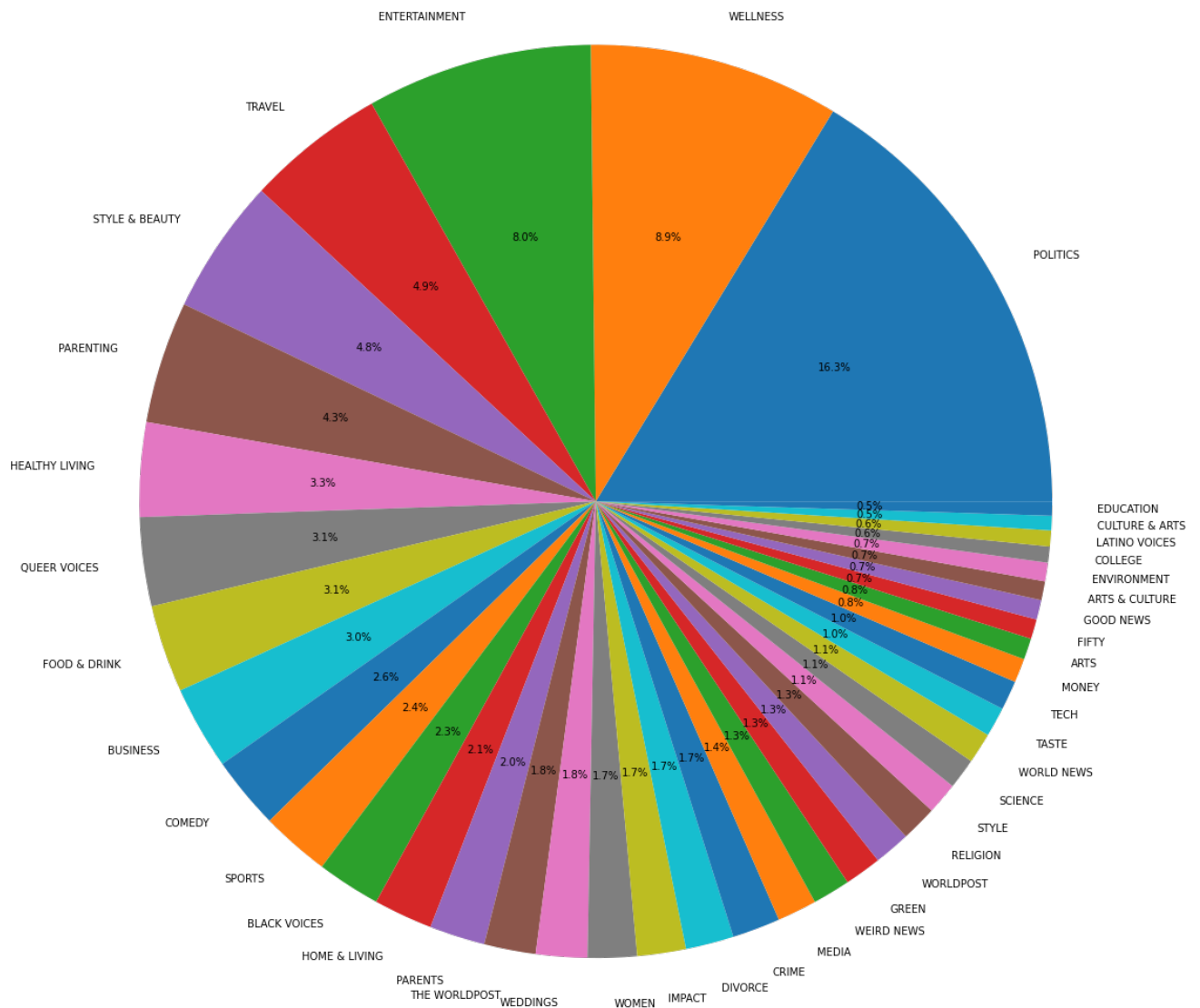
```
Out[48]:
```

POLITICS	32739
WELLNESS	17827
ENTERTAINMENT	16058
TRAVEL	9887
STYLE & BEAUTY	9649
PARENTING	8677
HEALTHY LIVING	6694
QUEER VOICES	6314
FOOD & DRINK	6226
BUSINESS	5937
COMEDY	5175
SPORTS	4884
BLACK VOICES	4528
HOME & LIVING	4195
PARENTS	3955
THE WORLDPOST	3664
WEDDINGS	3651
WOMEN	3490
IMPACT	3459
DIVORCE	3426
CRIME	3405
MEDIA	2815
WEIRD NEWS	2670
GREEN	2622
WORLDPOST	2579
RELIGION	2556
STYLE	2254
SCIENCE	2178
WORLD NEWS	2177
TASTE	2096
TECH	2082
MONEY	1707
ARTS	1509
FIFTY	1401
GOOD NEWS	1398
ARTS & CULTURE	1339
ENVIRONMENT	1323
COLLEGE	1144
LATINO VOICES	1129
CULTURE & ARTS	1030
EDUCATION	1004

Name: category, dtype: int64

```
In [49]:
```

```
fig = plt.figure(figsize=(20,20))
plt.pie(dataset['category'].value_counts().values,
        labels=dataset['category'].value_counts().index,
        autopct='%1.1f%%');
```



```
In [50]: categories = dataset['category'].value_counts().index

def groupper(grouplist,name):
    for ele in categories:
        if ele in grouplist:
            dataset.loc[dataset['category'] == ele, 'category'] = name
```

```
In [51]: groupper( grouplist= ['WELLNESS', 'HEALTHY LIVING'] , name = 'HEALTH AND WEL

groupper( grouplist= ['SPORTS','ENTERTAINMENT' , 'COMEDY','WEIRD NEWS','ARTS'

groupper( grouplist= ['BUSINESS' , 'MONEY','TECH', 'SCIENCE'] , name = 'BUS

groupper( grouplist= ['PARENTING', 'PARENTS' , 'EDUCATION' , 'COLLEGE', 'TRAVEL'
```

```
In [52]: # Shuffle the Dataset.
shuffled_df = dataset.sample(frac=1,random_state=4)
# Put each category in a separate dataset.
politics = shuffled_df.loc[shuffled_df['category'] == 'POLITICS']
sports = shuffled_df.loc[shuffled_df['category'] == 'SPORTS AND ENTERTAINMENT']
health = shuffled_df.loc[shuffled_df['category'] == 'HEALTH AND WELLNESS']
businessandscience = shuffled_df.loc[shuffled_df['category'] == 'BUSINESS AND SCIENCE']
general = shuffled_df.loc[shuffled_df['category'] == 'GENERAL']
#Randomly select 15000 observations from majority categories
politics_normal = shuffled_df.loc[shuffled_df['category'] == 'POLITICS'].sample(15000)
sports_normal = shuffled_df.loc[shuffled_df['category'] == 'SPORTS AND ENTERTAINMENT'].sample(15000)
health_normal = shuffled_df.loc[shuffled_df['category'] == 'HEALTH AND WELLNESS'].sample(15000)
# Concatenate all dataframes again
normalized_df = pd.concat([politics_normal , sports_normal , health_normal , businessandscience , general])
```

In [53]: `normalized_df`

Out[53]:

	category	headline	short_description
87800	POLITICS	Letter to My Sons on Father's Day	That's what inspired President Obama to launch...
20916	POLITICS	Exxon Fires Back Over Fine For Violating Russi...	Exxon sued the government over the \$2 million ...
5562	POLITICS	Donald Trump Puts Focus On Mental Health In Fi...	He didn't use the word "gun" once.
33767	POLITICS	National Security Council Spokesman Resigns Ov...	Edward Price said the \n"final straw" was Trum...
1498	POLITICS	Michigan Gov. Candidate Delayed Rescue Of Anim...	Shri Thanedar fought an effort to place over 1...
...	...	...	...
119370	BUSINESS AND SCIENCE	The Reason Gap Workers Are Wearing White This ...	
196283	BUSINESS AND SCIENCE	Slainte! Ireland From Space Is Shamrock Green ...	While vegetation is abundant, little of it is ...
168414	BUSINESS AND SCIENCE	Parasitic Worm VIDEO Shows Huge Nematode Emerg...	When YouTube user Brent Askwith saw a freakish...
63346	BUSINESS AND SCIENCE	The Story Behind The Craziest Corporate Collap...	This is madness.
177465	BUSINESS AND SCIENCE	Faces Of The Great Depression (PHOTOS)	The unemployment rate fell below 8 percent for...

56904 rows × 3 columns

In [54]: `normalized_df.duplicated().sum()` *#total duplicates*

Out[54]: 59

In [55]: `normalized_df.drop_duplicates(keep='last', inplace=True)`

In [56]: `normalized_df.loc[normalized_df['headline'] == "", 'headline'] = np.nan
normalized_df.dropna(subset=['headline'], inplace=True)
print(len(normalized_df[normalized_df['headline'] == ""]))`

0

In [57]: `normalized_df.loc[normalized_df['short_description'] == "", 'short_descriptio
normalized_df.dropna(subset=['short_description'], inplace=True)
print(len(normalized_df[normalized_df['short_description'] == ""]))`

0

In [58]: `normalized_df['text'] = normalized_df['headline']+" "+normalized_df['short_de
normalized_df.drop(columns =['headline','short_description'],axis = 1, inplac
normalized_df.head()`

Out[58]:

	category	text
87800	POLITICS	Letter to My Sons on Father's Day That's what ...
20916	POLITICS	Exxon Fires Back Over Fine For Violating Russi...
5562	POLITICS	Donald Trump Puts Focus On Mental Health In Fi...
33767	POLITICS	National Security Council Spokesman Resigns Ov...
1498	POLITICS	Michigan Gov. Candidate Delayed Rescue Of Anim...

In [59]:

```
from sklearn.utils import shuffle
normalized_df = shuffle(normalized_df)
normalized_df.reset_index(inplace=True, drop=True)
```

In [60]:

```
stop = set(stopwords.words('english'))
punctuation = list(string.punctuation)
stop.update(punctuation)

def strip_html(text):
    soup = BeautifulSoup(text, "html.parser")
    return soup.get_text()
def remove_between_square_brackets(text):
    return re.sub(r'\[[^\]]*\]', '', text)
def remove_urls(text):
    return re.sub(r'http\S+', '', text)
def remove_stopwords(text):
    final_text = []
    for i in text.split():
        if i.strip().lower() not in stop:
            final_text.append(i.strip())
    return " ".join(final_text)
def clean_text(text):
    text = re.sub(r'[\.\*\?\\]', '', text)
    text = re.sub(r'https?://\S+|www\.\S+', '', text)
    text = re.sub(r'<.*?>+', '', text)
    text = re.sub(r'[%s]' % re.escape(string.punctuation), '', text)
    text = re.sub(r'\n', '', text)
    text = re.sub(r'\w*\d\w*', '', text)
    text = re.sub(r'caption','',text)
    text = re.sub(r'photo','',text)
    text = re.sub(r'containerid','',text)
    text = re.sub(r'function','',text)
    text = re.sub(r'modernizr','',text)
    text = re.sub(r'modernizrphone','',text)
    text = re.sub(r'modernizrmobile','',text)
    text = re.sub(r'modernizrtablet','',text)
    text = re.sub(r'typeof','',text)
    text = re.sub(r'videopinner','',text)
    text = re.sub(r'undefined','',text)
    text = re.sub(r'null','',text)
    text = str(text).lower()
    return text

#Removing the noisy text
def denoise_text(text):
    text = strip_html(text)
    text = remove_between_square_brackets(text)
    text = remove_urls(text)
    text = remove_stopwords(text)
    text = clean_text(text)
    return text
```

In [61]:

```
normalized_df['text']=normalized_df['text'].apply(denoise_text)
```

In [62]:

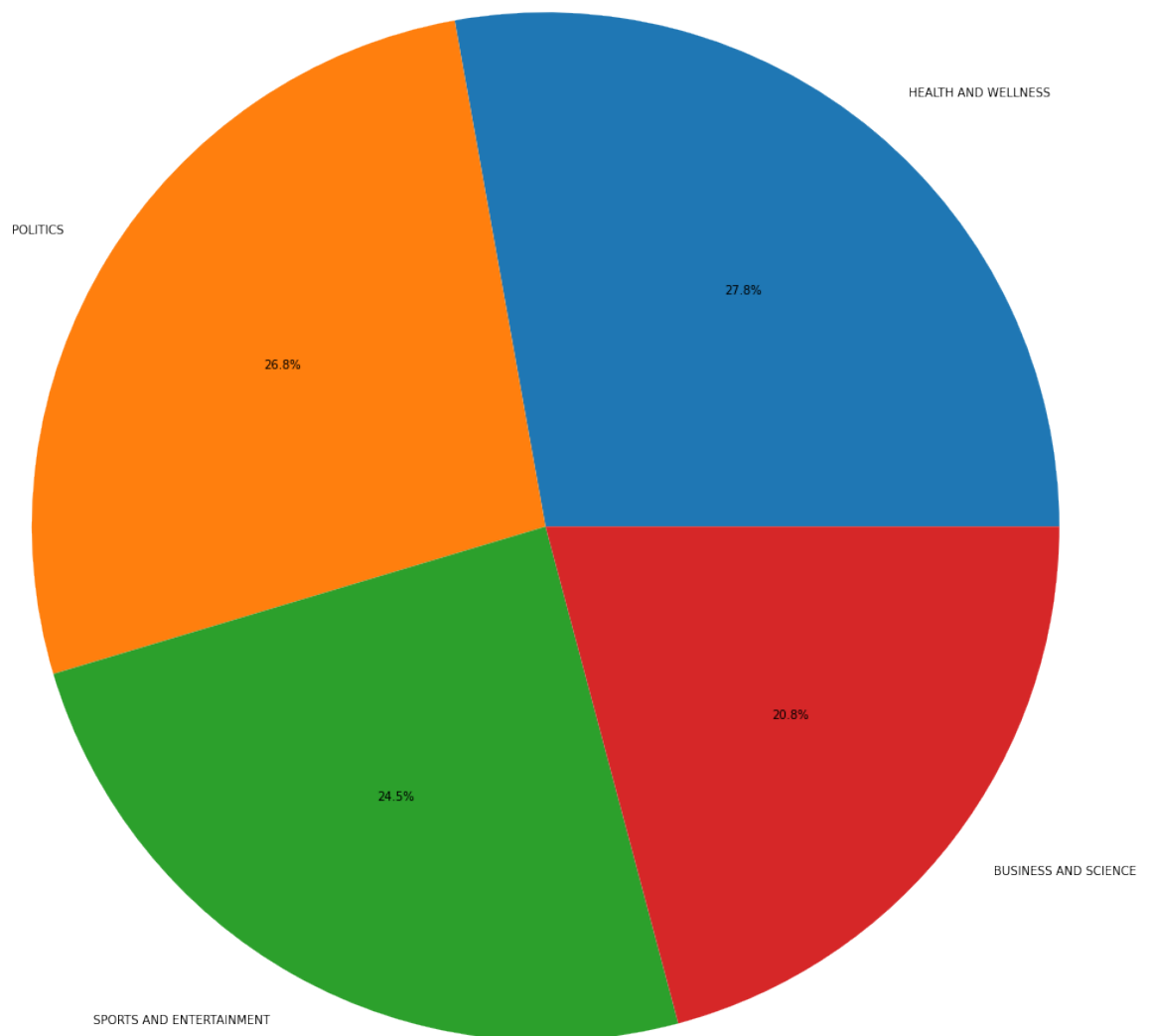
```
dataset = normalized_df
```

```
In [63]: print("We have a total of {} categories now".format(dataset['category'].nunique()))
dataset['category'].value_counts()
```

We have a total of 4 categories now

```
Out[63]: HEALTH AND WELLNESS      14140
POLITICS                        13613
SPORTS AND ENTERTAINMENT       12464
BUSINESS AND SCIENCE           10588
Name: category, dtype: int64
```

```
In [64]: fig = plt.figure(figsize=(20,20))
plt.pie(dataset['category'].value_counts().values,
        labels=dataset['category'].value_counts().index,
        autopct='%1.1f%%');
```



```
In [65]: df = dataset.copy() #Let's create a copy of the data frame
```

```
In [66]: from sklearn.utils import shuffle
df = shuffle(df)
df.reset_index(inplace=True, drop=True)
```

```
In [67]: tokenizer = Tokenizer()
tokenizer.fit_on_texts(df.text)
X = tokenizer.texts_to_sequences(df.text)
df['words'] = X
```

```
In [68]: df.drop(columns=['text'],axis = 1, inplace=True)
df
```

Out[68]:

	category	words
0	HEALTH AND WELLNESS	[296, 4150, 23336, 43, 31, 2137, 7986, 577, 36...
1	POLITICS	[45, 1606, 1119, 1063, 92, 2098, 5606, 7369, 2...
2	SPORTS AND ENTERTAINMENT	[208, 376, 19462, 3786, 171, 193, 1910, 77, 15...
3	POLITICS	[2547, 6827, 183, 19463, 45, 505, 1927, 915, 5...
4	POLITICS	[120, 19, 926, 594, 46, 19464, 13727, 9651, 17...
...	...	...
50800	BUSINESS AND SCIENCE	[6026, 25288, 1301, 387, 17817, 3680, 10589, 4...
50801	BUSINESS AND SCIENCE	[180, 18621, 1094, 2930, 4040, 835, 695, 75, 2...
50802	HEALTH AND WELLNESS	[6551, 146, 286, 881, 9536, 13716, 2509, 1087,...
50803	POLITICS	[29, 45, 934, 5794, 280, 6029, 7, 772, 2, 33, ...
50804	POLITICS	[4154, 3791, 3133, 4649, 796, 2403, 6688, 6688...

50805 rows × 2 columns

```
In [69]: maxlen = 50
X = list(sequence.pad_sequences(df.words, maxlen=maxlen))
```

```
In [70]: categories = df.groupby('category').size().index.tolist()
category_int = {}
int_category = {}
for i, k in enumerate(categories):
    category_int.update({k:i})
    int_category.update({i:k})

df['c2id'] = df['category'].apply(lambda x: category_int[x])
```

```
In [71]: category_int
```

Out[71]: {'BUSINESS AND SCIENCE': 0,
'HEALTH AND WELLNESS': 1,
'POLITICS': 2,
'SPORTS AND ENTERTAINMENT': 3}

```
In [72]: word_index = tokenizer.word_index

EMBEDDING_DIM = 100

embeddings_index = {}
f = open('/Users/moe/Desktop/glove/glove.6B.100d.txt')
for line in f:
    values = line.split()
    word = values[0]
    coefs = np.asarray(values[1:], dtype='float32')
    embeddings_index[word] = coefs
f.close()

print('Found %s unique tokens.' % len(word_index))
print('Total %s word vectors.' % len(embeddings_index))

Found 56491 unique tokens.
Total 400000 word vectors.
```



```
In [73]: embedding_matrix = np.zeros((len(word_index) + 1, EMBEDDING_DIM))
        for word, i in word_index.items():
            embedding_vector = embeddings_index.get(word)
            if embedding_vector is not None:
                embedding_matrix[i] = embedding_vector

        embedding_layer = Embedding(len(word_index)+1,
                                    EMBEDDING_DIM,
                                    embeddings_initializer=Constant(embedding_matrix),
                                    input_length=maxlen,
                                    trainable=False)
```

```
In [74]: # prepared data

X = np.array(X)
Y = np_utils.to_categorical(list(df.c2id))

# and split to training set and validation set

seed = 29
x_train, x_val, y_train, y_val = train_test_split(X, Y, test_size=0.2, random
```

```
In [75]: batch_size = 256
        epochs = 22
        embed_size = 100
        max_features = 56492
```

```
In [76]: learning_rate_reduction = ReduceLROnPlateau(monitor='val_accuracy', patience =
```

```
In [77]: model = Sequential()
        #Non-trainable embeddidng layer
        model.add(Embedding(max_features, output_dim=embed_size, weights=[embedding_m
        #LSTM
        model.add(LSTM(units=128 , return_sequences = True , recurrent_dropout = 0.25
        model.add(LSTM(units=64 , recurrent_dropout = 0.1 , dropout = 0.1))
        model.add(Dense(units = 32 , activation = 'relu'))
        model.add(Dense(4, activation = 'sigmoid'))
        model.compile(optimizer=keras.optimizers.Adam(learning_rate = 0.01), loss='bi
        model.summary()
```

Model: "sequential_1"

Layer (type)	Output Shape	Param #
=====		
embedding_3 (Embedding)	(None, 50, 100)	5649200

lstm_2 (LSTM)	(None, 50, 128)	117248

lstm_3 (LSTM)	(None, 64)	49408

dense_2 (Dense)	(None, 32)	2080

dense_3 (Dense)	(None, 4)	132
=====		
Total params: 5,818,068		
Trainable params: 168,868		
Non-trainable params: 5,649,200		

```
In [78]: history = model.fit(x_train, y_train, batch_size = batch_size , validation_da

Epoch 1/22
159/159 [=====] - 83s 501ms/step - loss: 0.2846 - acc
uracy: 0.7587 - val_loss: 0.2204 - val_accuracy: 0.8243
Epoch 2/22
159/159 [=====] - 96s 607ms/step - loss: 0.2103 - acc
uracy: 0.8327 - val_loss: 0.2086 - val_accuracy: 0.8319
Epoch 3/22
```

159/159 [=====] - 93s 585ms/step - loss: 0.1918 - accuracy: 0.8479 - val_loss: 0.1978 - val_accuracy: 0.8400
Epoch 4/22
159/159 [=====] - 95s 600ms/step - loss: 0.1781 - accuracy: 0.8602 - val_loss: 0.2092 - val_accuracy: 0.8357
Epoch 5/22
159/159 [=====] - 96s 606ms/step - loss: 0.1690 - accuracy: 0.8669 - val_loss: 0.1995 - val_accuracy: 0.8395

Epoch 00005: ReduceLROnPlateau reducing learning rate to 0.004999999888241291.
Epoch 6/22
159/159 [=====] - 85s 537ms/step - loss: 0.1528 - accuracy: 0.8805 - val_loss: 0.1967 - val_accuracy: 0.8457
Epoch 7/22
159/159 [=====] - 84s 526ms/step - loss: 0.1434 - accuracy: 0.8890 - val_loss: 0.2007 - val_accuracy: 0.8488
Epoch 8/22
159/159 [=====] - 83s 522ms/step - loss: 0.1380 - accuracy: 0.8932 - val_loss: 0.2006 - val_accuracy: 0.8461
Epoch 9/22
159/159 [=====] - 83s 520ms/step - loss: 0.1326 - accuracy: 0.8972 - val_loss: 0.2038 - val_accuracy: 0.8467

Epoch 00009: ReduceLROnPlateau reducing learning rate to 0.0024999999441206455.
Epoch 10/22
159/159 [=====] - 85s 533ms/step - loss: 0.1219 - accuracy: 0.9059 - val_loss: 0.2044 - val_accuracy: 0.8464
Epoch 11/22
159/159 [=====] - 84s 527ms/step - loss: 0.1172 - accuracy: 0.9100 - val_loss: 0.2155 - val_accuracy: 0.8447

Epoch 00011: ReduceLROnPlateau reducing learning rate to 0.0012499999720603228.
Epoch 12/22
159/159 [=====] - 84s 526ms/step - loss: 0.1099 - accuracy: 0.9165 - val_loss: 0.2160 - val_accuracy: 0.8468
Epoch 13/22
159/159 [=====] - 93s 588ms/step - loss: 0.1091 - accuracy: 0.9149 - val_loss: 0.2202 - val_accuracy: 0.8461

Epoch 00013: ReduceLROnPlateau reducing learning rate to 0.0006249999860301614.
Epoch 14/22
159/159 [=====] - 92s 579ms/step - loss: 0.1040 - accuracy: 0.9187 - val_loss: 0.2203 - val_accuracy: 0.8457
Epoch 15/22
159/159 [=====] - 93s 586ms/step - loss: 0.1029 - accuracy: 0.9205 - val_loss: 0.2212 - val_accuracy: 0.8464

Epoch 00015: ReduceLROnPlateau reducing learning rate to 0.0003124999930150807.
Epoch 16/22
159/159 [=====] - 92s 582ms/step - loss: 0.0991 - accuracy: 0.9240 - val_loss: 0.2249 - val_accuracy: 0.8474
Epoch 17/22
159/159 [=====] - 93s 583ms/step - loss: 0.0999 - accuracy: 0.9231 - val_loss: 0.2230 - val_accuracy: 0.8475

Epoch 00017: ReduceLROnPlateau reducing learning rate to 0.00015624999650754035.
Epoch 18/22
159/159 [=====] - 91s 571ms/step - loss: 0.0989 - accuracy: 0.9234 - val_loss: 0.2247 - val_accuracy: 0.8461
Epoch 19/22
159/159 [=====] - 89s 561ms/step - loss: 0.0990 - accuracy: 0.9246 - val_loss: 0.2248 - val_accuracy: 0.8460

Epoch 00019: ReduceLROnPlateau reducing learning rate to 7.812499825377017e-05.
Epoch 20/22
159/159 [=====] - 88s 553ms/step - loss: 0.0984 - accuracy: 0.9239 - val_loss: 0.2252 - val_accuracy: 0.8463
Epoch 21/22
159/159 [=====] - 86s 540ms/step - loss: 0.0981 - accuracy: 0.9236 - val_loss: 0.2252 - val_accuracy: 0.8461

Epoch 00021: ReduceLROnPlateau reducing learning rate to 3.9062499126885086e-05.

Epoch 22/22
159/159 [=====] - 87s 548ms/step - loss: 0.0965 - accuracy: 0.9246 - val_loss: 0.2261 - val_accuracy: 0.8468

```
In [79]: print("Accuracy of the model on Training Data is - " , model.evaluate(x_train, y_train))
print("Accuracy of the model on Testing Data is - " , model.evaluate(x_val, y_val))

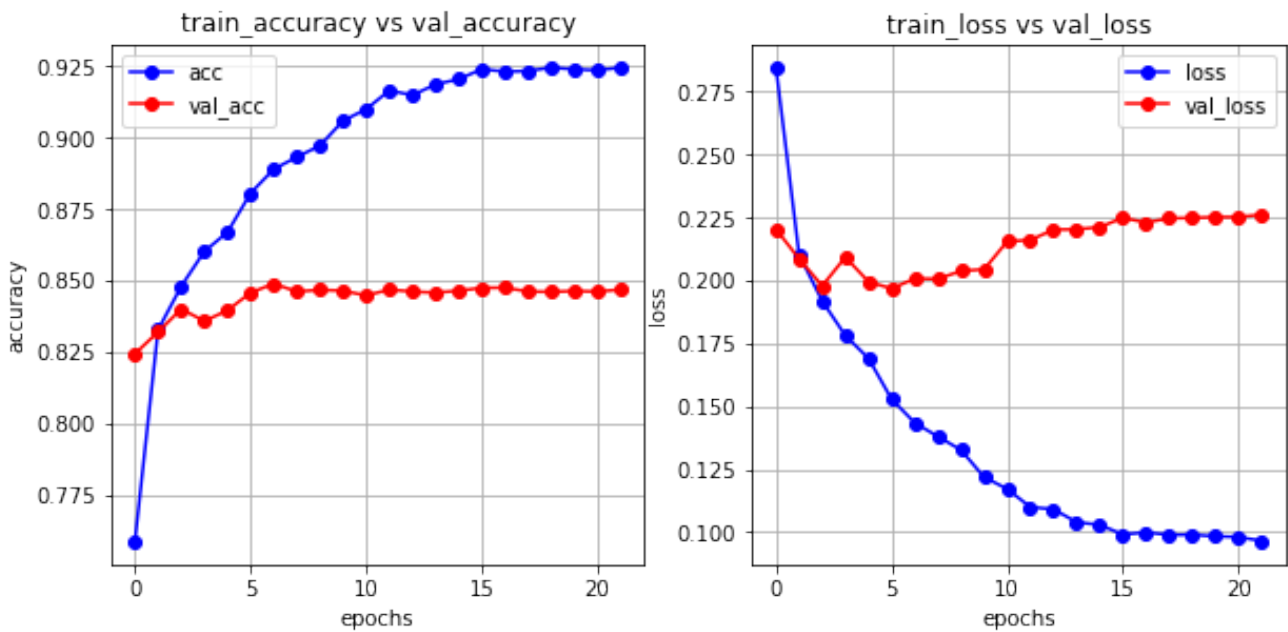
1271/1271 [=====] - 32s 25ms/step - loss: 0.0605 - accuracy: 0.9585
Accuracy of the model on Training Data is - 95.84932327270508 %
318/318 [=====] - 8s 26ms/step - loss: 0.2261 - accuracy: 0.8468
Accuracy of the model on Testing Data is - 84.67670679092407 %
```

```
In [80]: fig = plt.figure(figsize=(10,10))

# Plot accuracy
plt.subplot(221)
plt.plot(history.history['accuracy'], 'bo-', label = "acc")
plt.plot(history.history['val_accuracy'], 'ro-', label = "val_acc")
plt.title("train_accuracy vs val_accuracy")
plt.ylabel("accuracy")
plt.xlabel("epochs")
plt.grid(True)
plt.legend()

# Plot loss function
plt.subplot(222)
plt.plot(history.history['loss'], 'bo-', label = "loss")
plt.plot(history.history['val_loss'], 'ro-', label = "val_loss")
plt.title("train_loss vs val_loss")
plt.ylabel("loss")
plt.xlabel("epochs")
plt.grid(True)
plt.legend()
```

Out[80]: <matplotlib.legend.Legend at 0x16b009430>



```
In [81]: y_pred=model.predict(x_val)
y_pred=np.argmax(y_pred, axis=1)
y_val=np.argmax(y_val, axis=1)
```

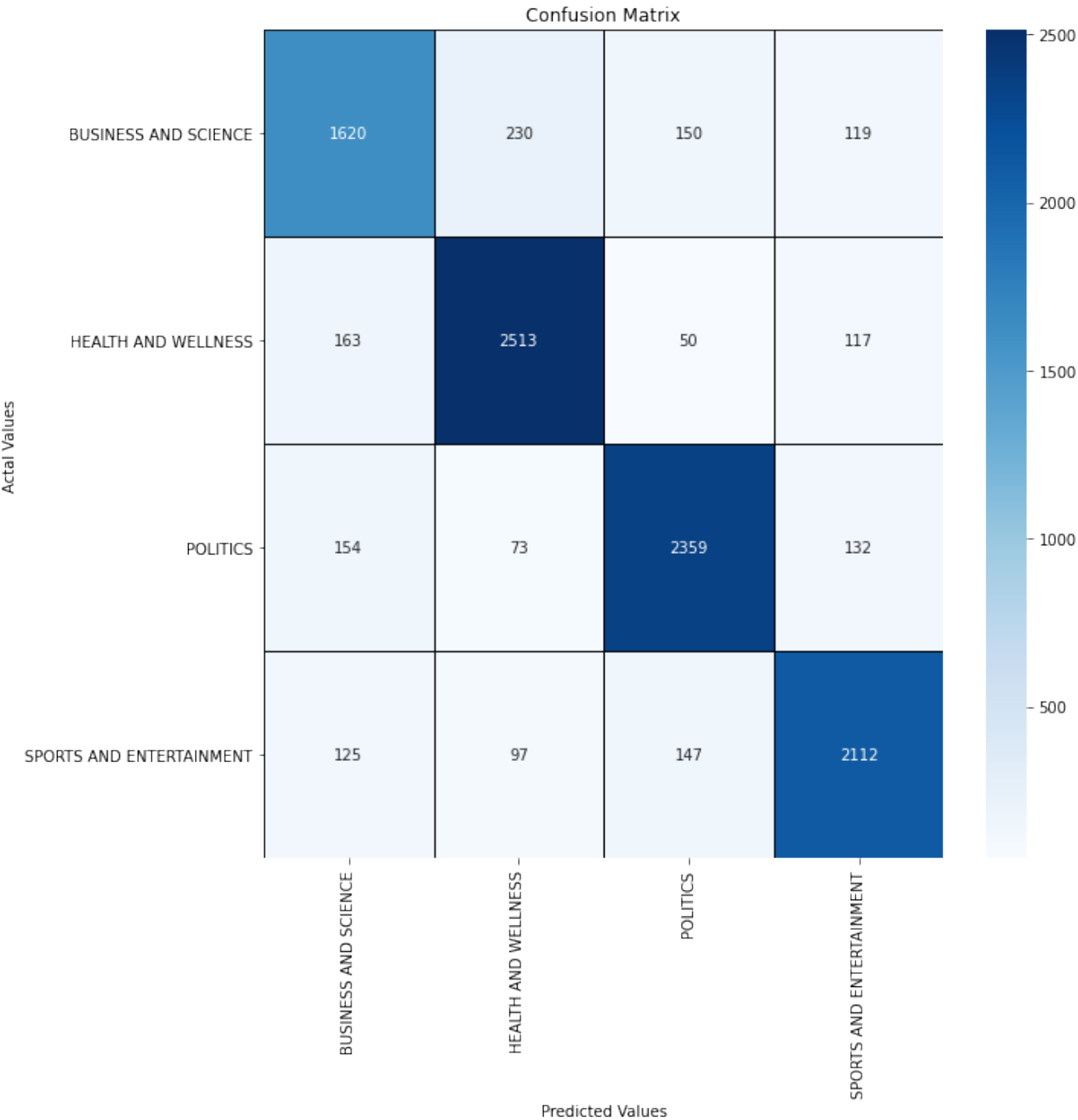
```
In [82]: print(classification_report(y_val, y_pred, target_names = ['BUSINESS AND SCIENCE', 'OTHER']))
```

	precision	recall	f1-score	support
BUSINESS AND SCIENCE	0.79	0.76	0.77	2119
HEALTH AND WELLNESS	0.86	0.88	0.87	2843
POLITICS	0.87	0.87	0.87	2718
SPORTS AND ENTERTAINMENT	0.85	0.85	0.85	2481
accuracy			0.85	10161
macro avg	0.84	0.84	0.84	10161
weighted avg	0.85	0.85	0.85	10161

```
In [83]: cm = confusion_matrix(y_val, y_pred)

cm_df = pd.DataFrame(cm,
                      index = ['BUSINESS AND SCIENCE', 'HEALTH AND WELLNESS', 'POLITICS', 'SPORTS AND ENTERTAINMENT'],
                      columns = ['BUSINESS AND SCIENCE', 'HEALTH AND WELLNESS', 'POLITICS', 'SPORTS AND ENTERTAINMENT'])

plt.figure(figsize=(10,10))
sns.heatmap(cm_df,cmap= "Blues", linecolor = 'black' , linewidth = 1 , annot = True)
plt.xticks(rotation=0)
plt.title('Confusion Matrix')
plt.ylabel('Actal Values')
plt.xlabel('Predicted Values')
plt.show()
```



```
In [43]: model.save_weights("trained_model.h5")
```

```
In [44]: import pickle
with open("tokenizer.pickle",mode="wb") as F:
    pickle.dump(tokenizer,F)
```

```
In [45]: import json
label_map = {1:'health',
             2:'politics',
             3:'sports and entertainment',
             0:'business and science',
             }

json.dump(label_map,open("label_map.json",mode="w"))
```

```
In [46]: class DeployModel():

    def __init__(self,weights_path,tokenizer_path,seq_length,label_map_path
                ):

        self.model = model
        self.model.load_weights(weights_path)
        self.tokenizer = pickle.load(open(tokenizer_path,mode="rb"))
        self.seq_len = seq_length
        self.label_map = json.load(open(label_map_path))

    def _prepare_data(self,text):

        cleaned = denoise_text(text)
        tokenized = self.tokenizer.texts_to_sequences([cleaned])
        padded = pad_sequences(tokenized,maxlen=self.seq_len)
        return padded

    def _predict(self,text):

        text = self._prepare_data(text)
        pred = int(self.model.predict_classes(text)[0])
        return str(pred)

    def result(self,text):

        pred = self._predict(text)
        return self.label_map[pred]
```

```
In [47]: deploy_model = DeployModel(weights_path="./trained_model.h5",
                                     tokenizer_path="./tokenizer.pickle",
                                     seq_length=maxlen,
                                     label_map_path="./label_map.json"
                                     )
```

```
In [48]: dfa = pd.read_csv('/Users/moe/Desktop/FINAL OUTPUT/ALLNEWSFAKE.csv')
```

```
In [49]: dfa['text']=dfa['text'].apply(denoise_text)
```

```
In [50]: for i in range(len(dfa)):
    r = deploy_model.result(dfa.at[i,'text'])
    dfa.at[i, 'category'] = r
```

```
/Users/moe/.pyenv/versions/3.8.10/envs/myvenv/lib/python3.8/site-packages/tensorflow/python/keras/engine/sequential.py:455: UserWarning: `model.predict_classes()` is deprecated and will be removed after 2021-01-01. Please use instead:
* `np.argmax(model.predict(x), axis=-1)`, if your model does multi-class classification (e.g. if it uses a `softmax` last-layer activation).
* `(model.predict(x) > 0.5).astype("int32")`, if your model does binary classification (e.g. if it uses a `sigmoid` last-layer activation).
warnings.warn("`model.predict_classes()` is deprecated and "
```

```
In [51]: dfa
```

Out[51]:		Date	Headline	Source	text	Result	category
	0	2020-04-16	Top 2020 prospect Jalen Green will head to NBA...	USAToday	top prospect jalen green head nba g league ins...	fake	sports and entertainment
	1	2020-04-28	Melissa McCarthy's dream about Mark Wahlberg m...	CNN	melissa mccarthys dream mark wahlberg may come...	fake	business and science
	2	2020-01-17	Biden Could Gain From Warren-Sanders Dispute	NewsMax	biden could gain warrensanders dispute former ...	fake	politics
	3	2020-08-23	Larry King Mourns Loss of Two of His Adult Chi...	NewsMax	larry king mourns loss two adult children with...	fake	health
	4	2020-03-01	A New York pizza shop is putting photos of dog...	CNN	new york pizza shop putting dogs pizza boxes h...	fake	business and science
	...	...	...	...	...	...	...
	24842	2020-06-24	NBA 2020 season restart: These players are exp...	USAToday	nba season restart players expected opt partic...	fake	sports and entertainment
	24843	2020-06-25	Resources by Reviewed: June 26	USAToday	resources reviewed june welcome lazy hazy days...	fake	health
	24844	2020-11-04	Fox's usual 'Big Noon Kickoff' team won't be o...	USAToday	foxs usual big noon kickoff team saturdays bro...	fake	business and science
	24845	2020-12-09	'Mars, here we come!!': SpaceX's Starship prot...	USAToday	mars come spacexs starship prototype performs ...	fake	business and science
	24846	25th August 2020	Pat McCluskey: Kenny Dalglish leads tributes t...	Scotsman	pat mccluskey kenny dalglish leads tributes fo...	fake	sports and entertainment

24847 rows x 6 columns

```
In [52]: dfa.to_csv(r'/Users/moe/Desktop/Scraped News/ALLNEWSFAKECATEGORY.csv' , index
```

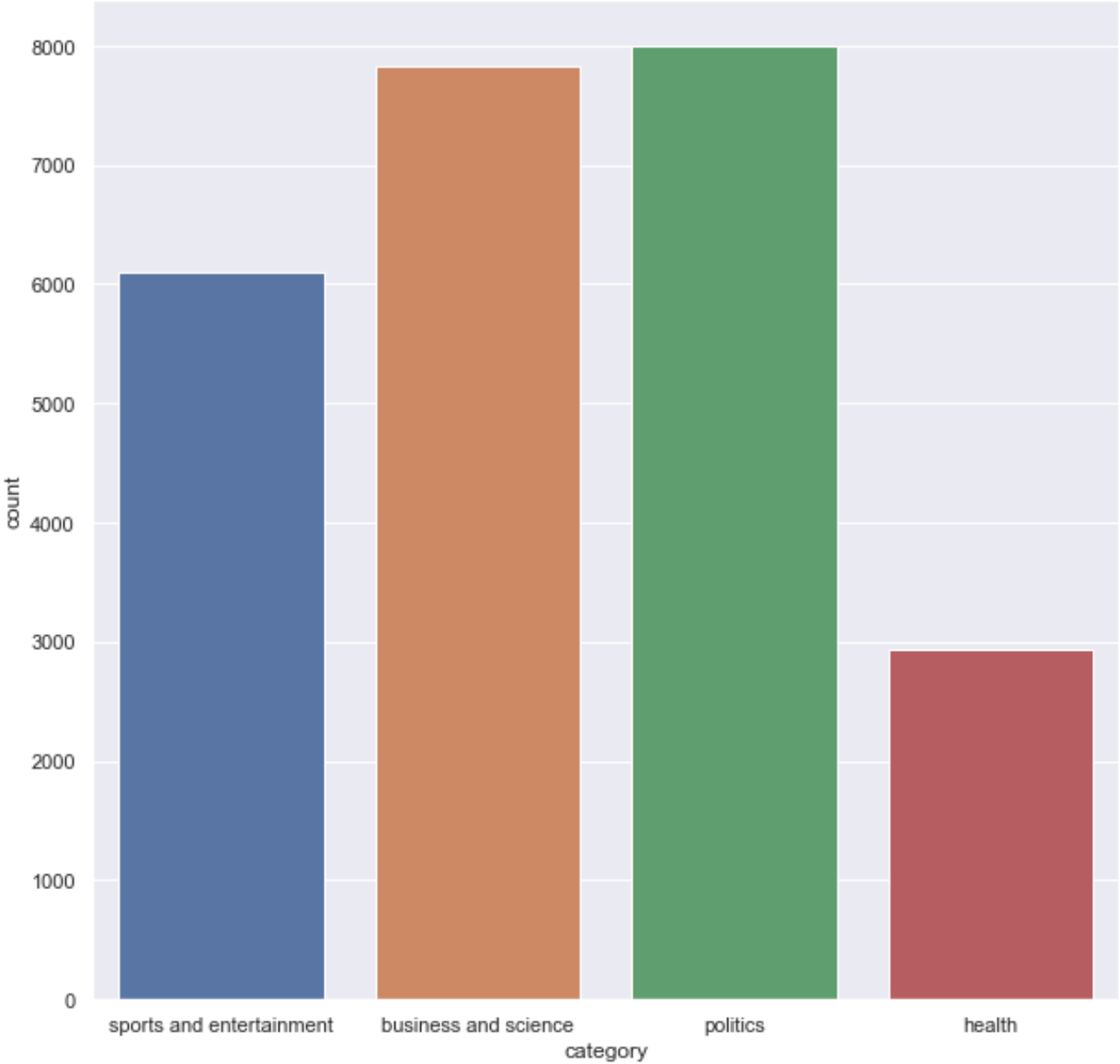
```
In [53]: dfa.category.value_counts()
```

politics 7991
business and science 7825
sports and entertainment 6097
health 2934
Name: category, dtype: int64

```
In [54]: sns.set_style("darkgrid")  
sns.set(rc = {'figure.figsize':(10,10)})  
sns.countplot(dfa.category)
```

```
/Users/moe/.pyenv/versions/3.8.10/envs/myvenv/lib/python3.8/site-packages/seaborn/_decorators.py:36: FutureWarning: Pass the following variable as a keyword arg: x. From version 0.12, the only valid positional argument will be `data`, and passing other arguments without an explicit keyword will result in an error or misinterpretation.
  warnings.warn(
```

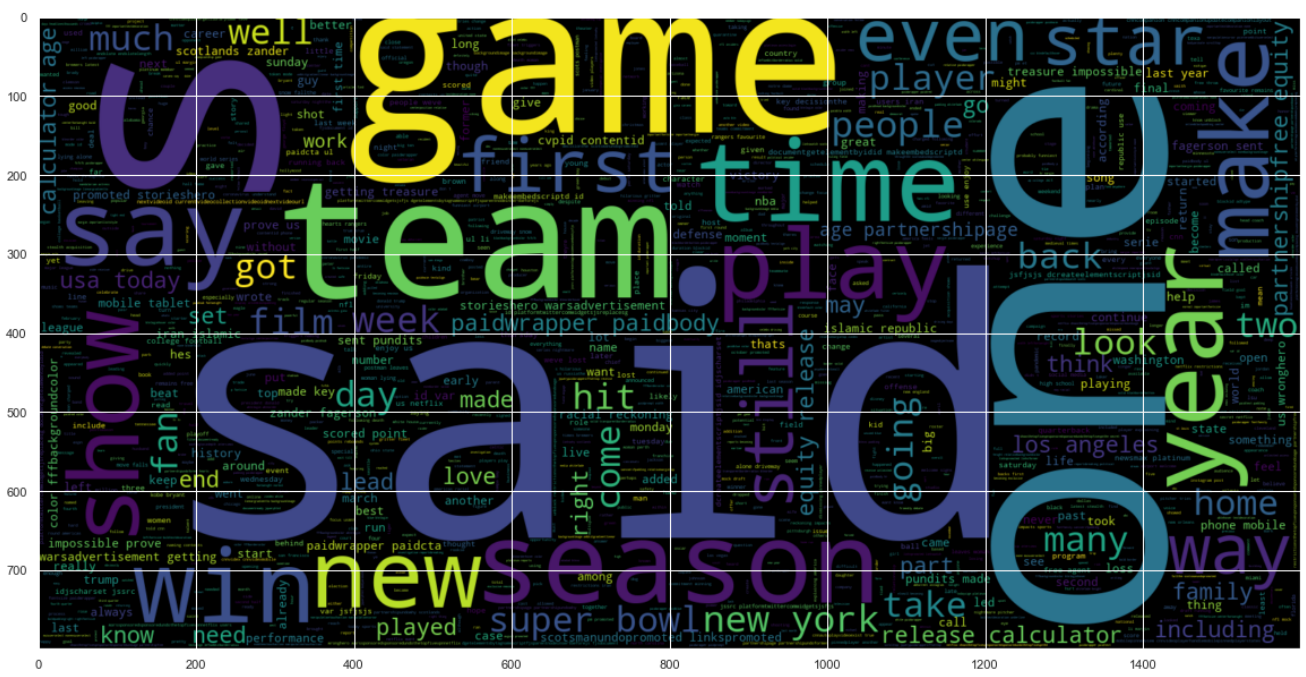
Out[54]: <AxesSubplot:xlabel='category', ylabel='count'>



Sports and entertainment word cloud

```
In [55]: plt.figure(figsize = (20,20))
wc = WordCloud(max_words = 2000 , width = 1600 , height = 800 , stopwords = S
plt.imshow(wc , interpolation = 'bilinear')
```

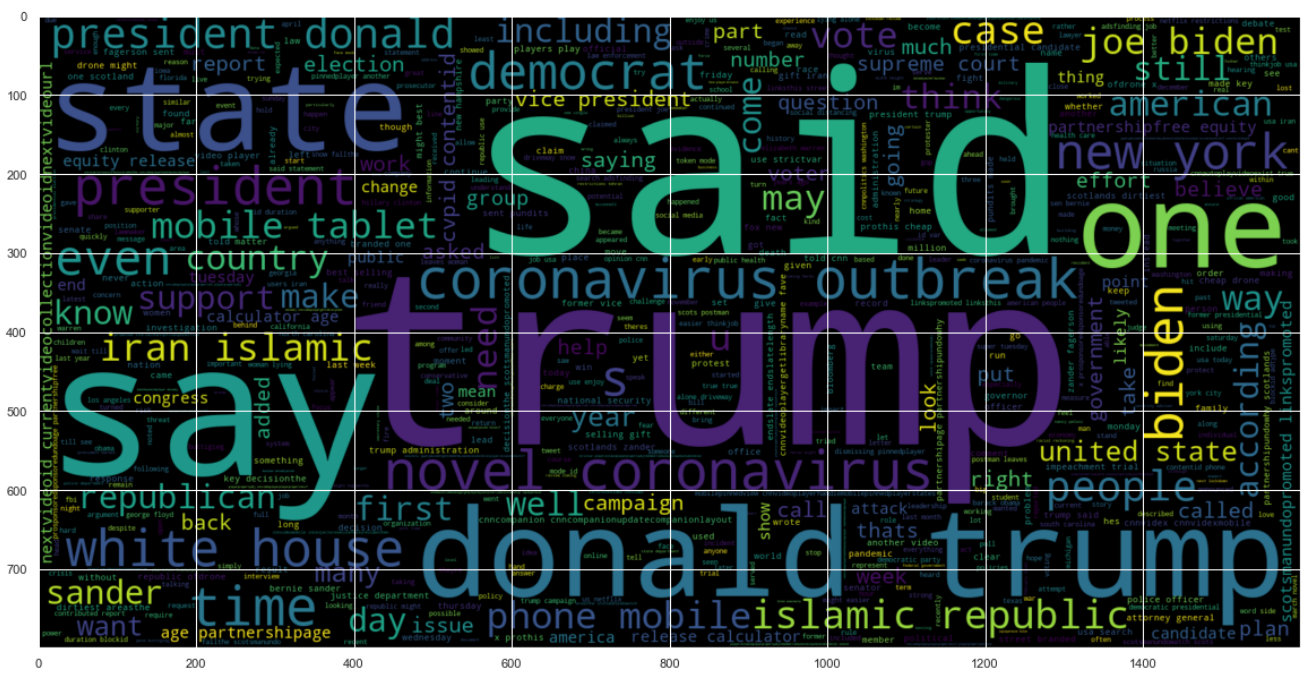

Out[55]: <matplotlib.image.AxesImage at 0x16d1a7550>



Politics word cloud

```
In [56]: plt.figure(figsize = (20,20))
wc = WordCloud(max_words = 2000 , width = 1600 , height = 800 , stopwords = S'
plt.imshow(wc , interpolation = 'bilinear')
```

Out[56]: <matplotlib.image.AxesImage at 0x16efdf0a0>

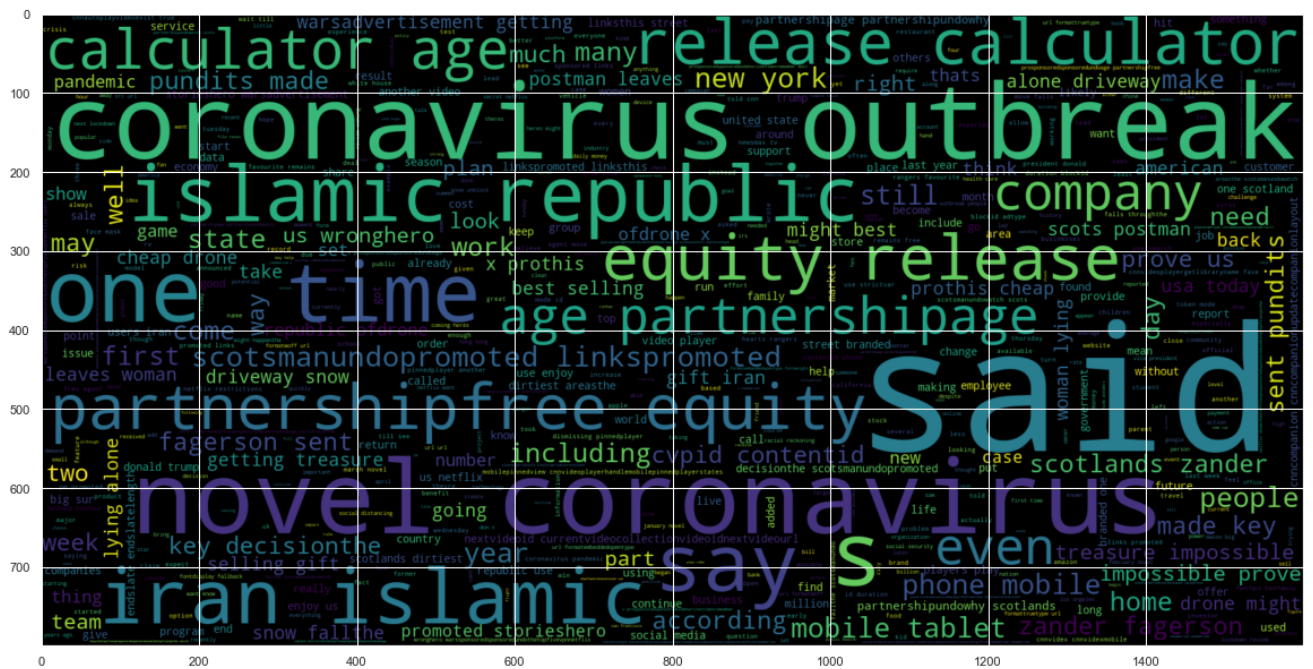


Business and science word cloud

```
In [57]: plt.figure(figsize = (20,20))
wc = WordCloud(max_words = 2000 , width = 1600 , height = 800 , stopwords = S'
plt.imshow(wc , interpolation = 'bilinear')
```



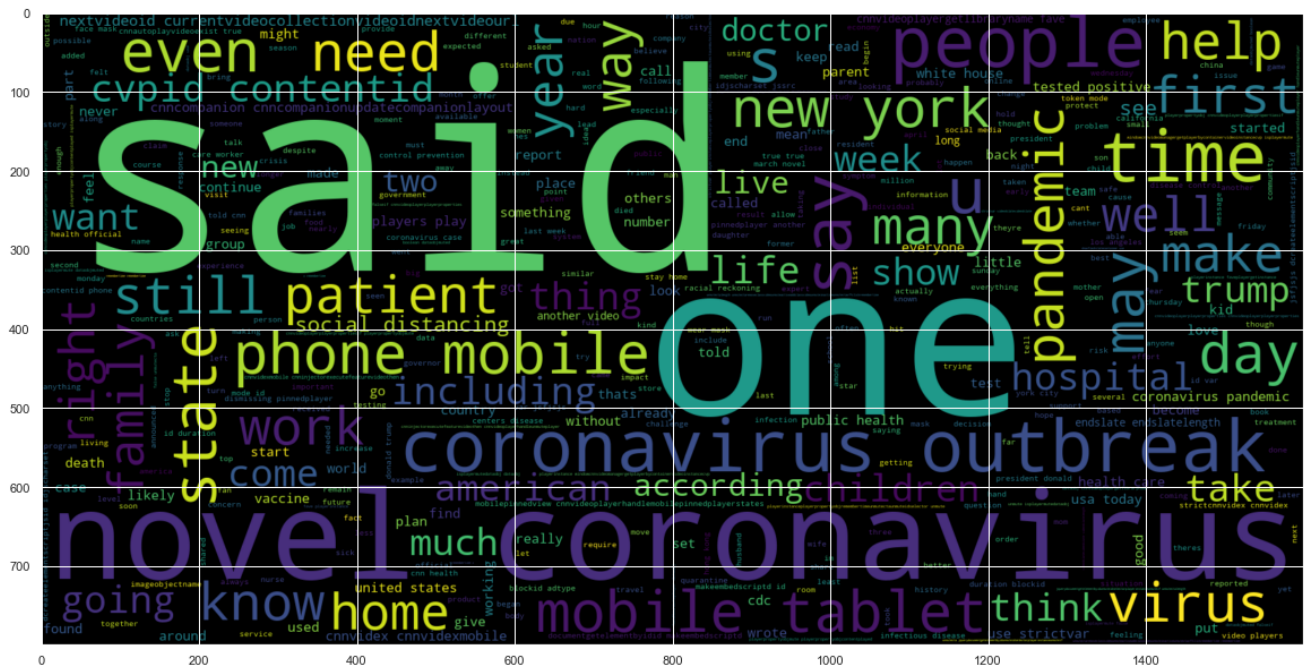
```
Out[57]: <matplotlib.image.AxesImage at 0x170217ca0>
```



Health word cloud

```
In [58]: plt.figure(figsize = (20,20))
wc = WordCloud(max_words = 2000 , width = 1600 , height = 800 , stopwords = S
plt.imshow(wc , interpolation = 'bilinear')
```

```
Out[58]: <matplotlib.image.AxesImage at 0x16fb3cb20>
```



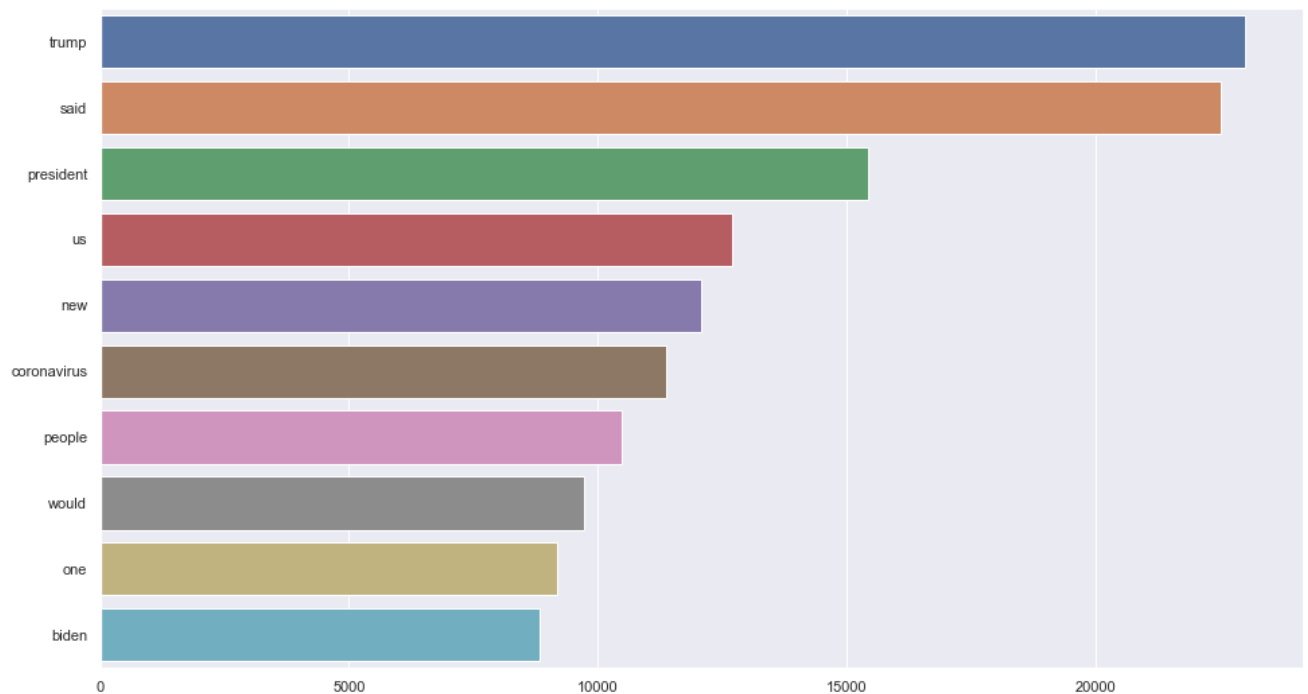
```
In [59]: dfapolitics = dfa.loc[dfa['category'] == 'politics']
dfasports = dfa.loc[dfa['category'] == 'sports and entertainment']
dfahealth = dfa.loc[dfa['category'] == 'health']
dfabusinessandscience = dfa.loc[dfa['category'] == 'business and science']
```

```
In [61]: from sklearn.feature_extraction.text import CountVectorizer
def get_top_text_ngrams(corpus, n, g):
    vec = CountVectorizer(ngram_range=(g, g)).fit(corpus)
    bag_of_words = vec.transform(corpus)
    sum_words = bag_of_words.sum(axis=0)
    words_freq = [(word, sum_words[0, idx]) for word, idx in vec.vocabulary_.items()]
    words_freq = sorted(words_freq, key = lambda x: x[1], reverse=True)
    return words_freq[:n]
```

Unigram, bigram and trigram analysis of politics

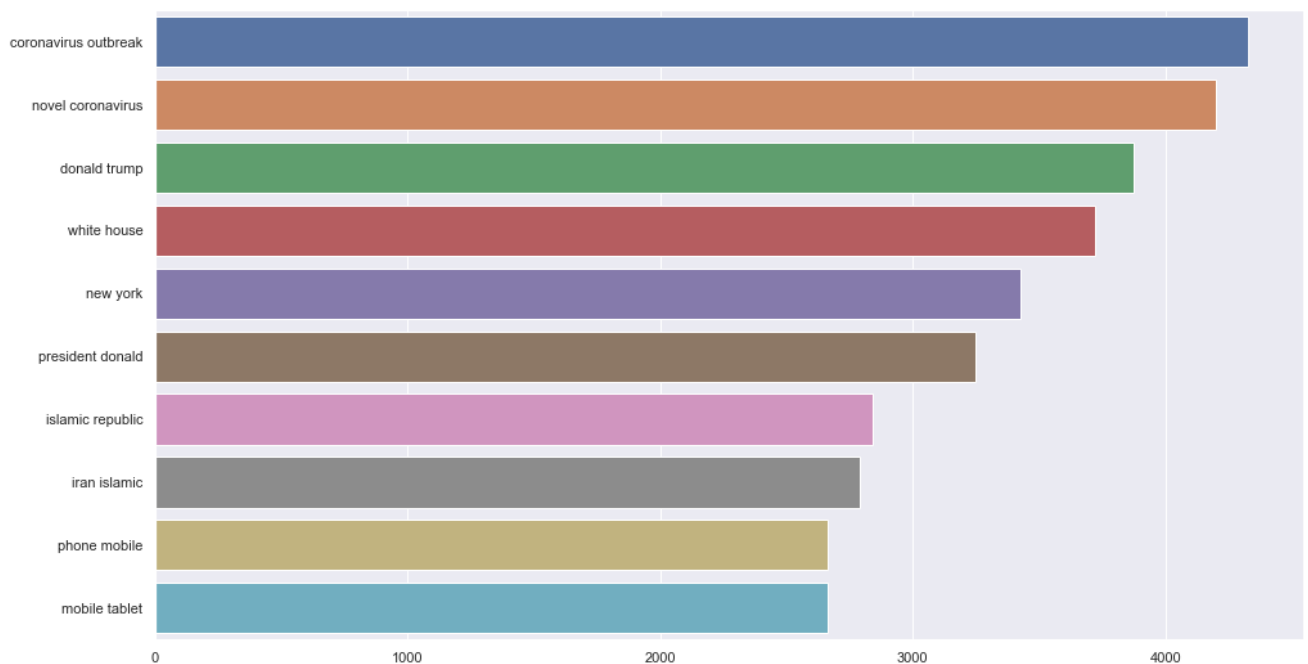
```
In [62]: plt.figure(figsize = (16,9))
most_common_uni = get_top_text_ngrams(dfapolitics.text,10,1)
most_common_uni = dict(most_common_uni)
sns.barplot(x=list(most_common_uni.values()),y=list(most_common_uni.keys()))
fig.suptitle('Unigram analysis of politics news 2020')
```

Out[62]: Text(0.5, 0.98, 'Unigram analysis of politics news 2020')



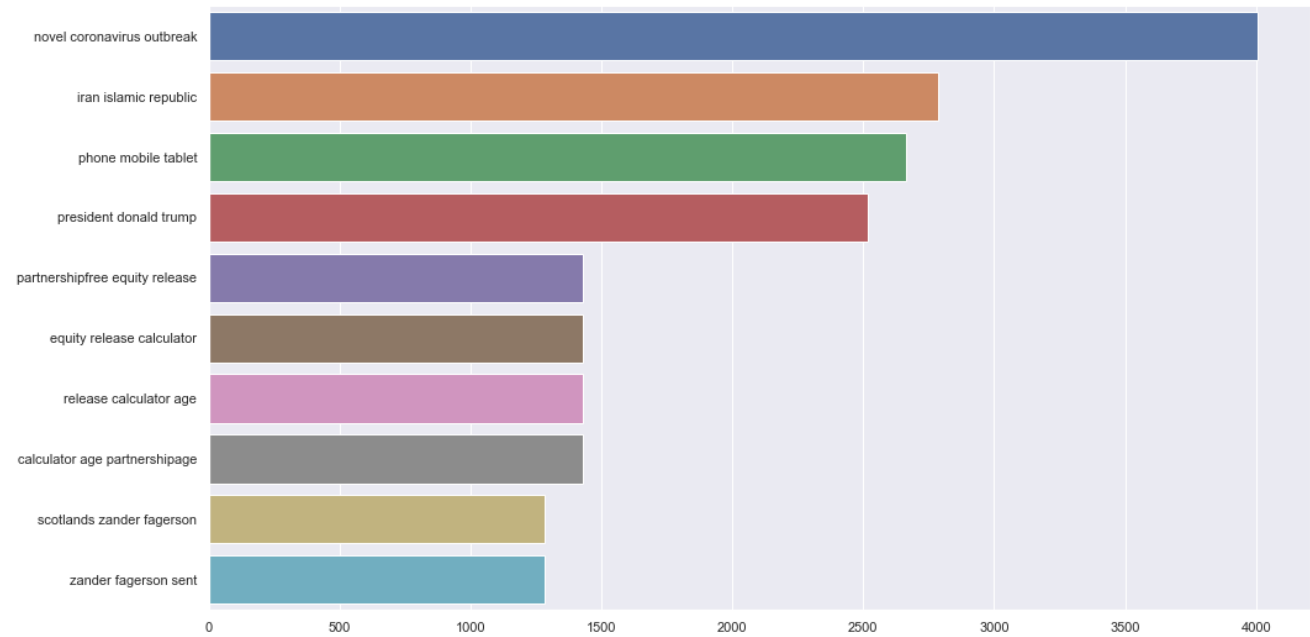
```
In [66]: plt.figure(figsize = (16,9))
most_common_bi = get_top_text_ngrams(dfapolitics.text,10,2)
most_common_bi = dict(most_common_bi)
sns.barplot(x=list(most_common_bi.values()),y=list(most_common_bi.keys()))
fig.suptitle('Bigram analysis of politics news 2020')
```

Out[66]: Text(0.5, 0.98, 'Bigram analysis of politics news 2020')



```
In [73]: plt.figure(figsize = (16,9))
most_common_tri = get_top_text_ngrams(dfapolitics.text,10,3)
most_common_tri = dict(most_common_tri)
sns.barplot(x=list(most_common_tri.values()),y=list(most_common_tri.keys()))
fig.suptitle('Trigram analysis of politics news 2020')
```

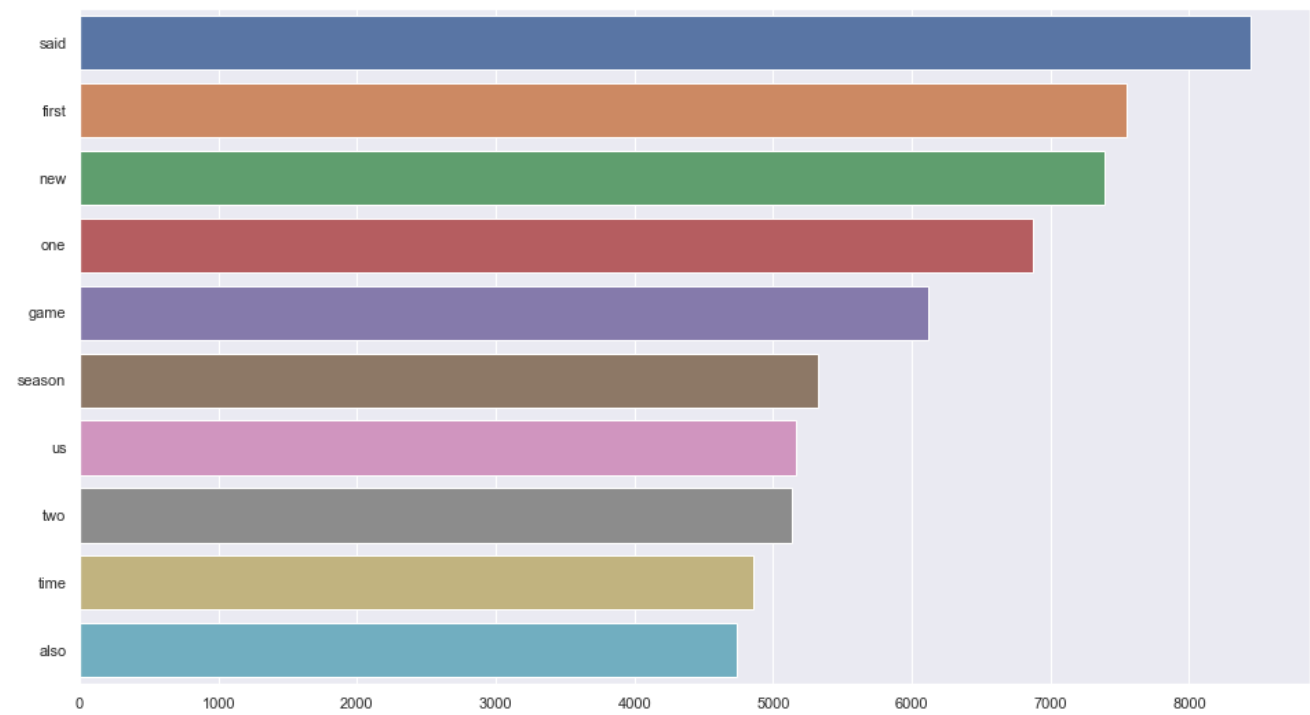
Out[73]: Text(0.5, 0.98, 'Trigram analysis of politics news 2020')



Unigram, bigram and trigram analysis of sports and entertainment

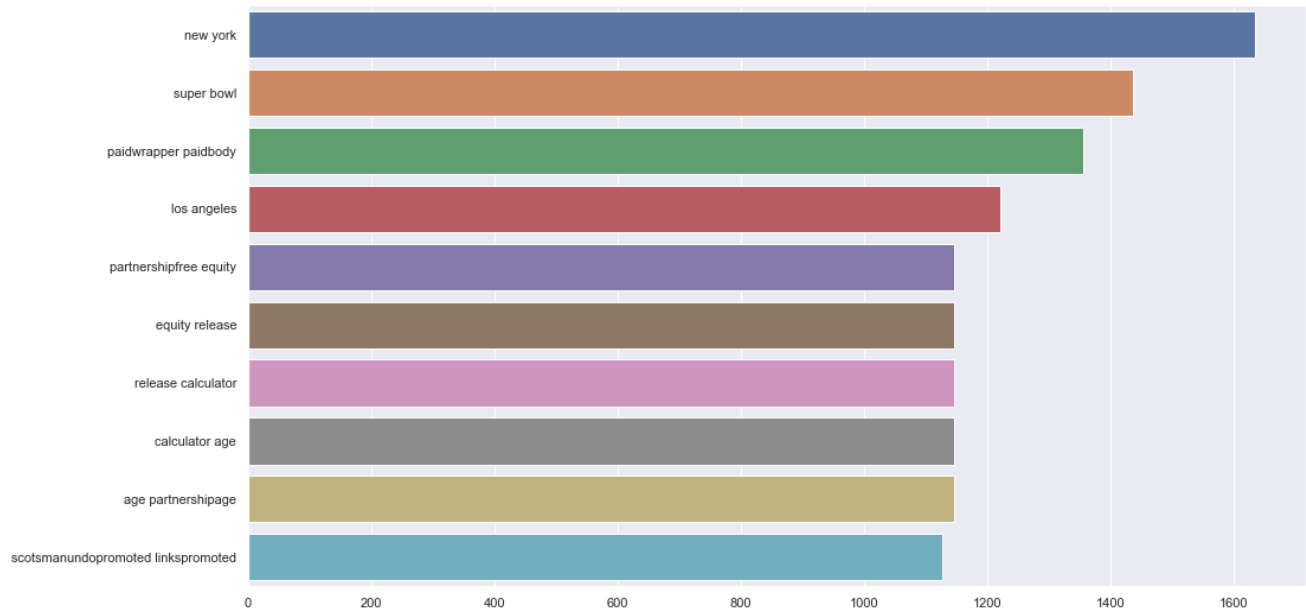
```
In [63]: plt.figure(figsize = (16,9))
most_common_uni = get_top_text_ngrams(dfasports.text,10,1)
most_common_uni = dict(most_common_uni)
sns.barplot(x=list(most_common_uni.values()),y=list(most_common_uni.keys()))
fig.suptitle('Unigram analysis of sports and entertainment news 2020')
```

Out[63]: Text(0.5, 0.98, 'Unigram analysis of sports and entertainment news 2020')



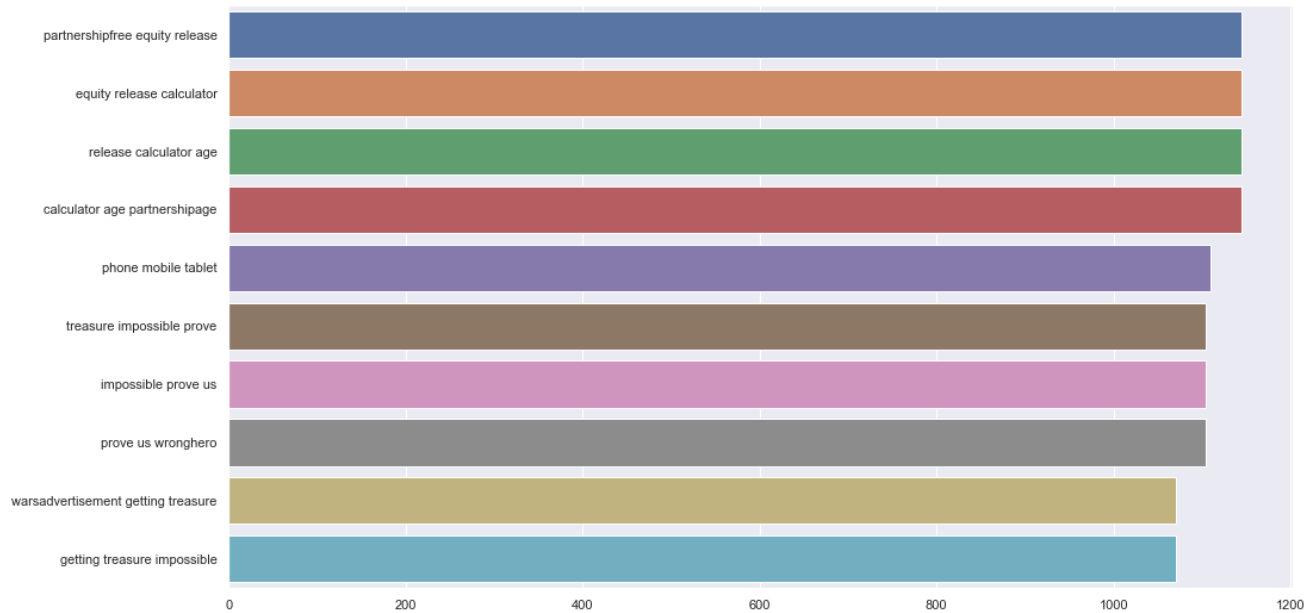
```
In [67]: plt.figure(figsize = (16,9))
most_common_bi = get_top_text_ngrams(dfasports.text,10,2)
most_common_bi = dict(most_common_bi)
sns.barplot(x=list(most_common_bi.values()),y=list(most_common_bi.keys()))
fig.suptitle('Bigram analysis of sports and entertainment news 2020')
```

Out[67]: Text(0.5, 0.98, 'Bigram analysis of sports and entertainment news 2020')



```
In [72]: plt.figure(figsize = (16,9))
most_common_tri = get_top_text_ngrams(dfasports.text,10,3)
most_common_tri = dict(most_common_tri)
sns.barplot(x=list(most_common_tri.values()),y=list(most_common_tri.keys()))
fig.suptitle('Trigram analysis of sports and entertainment news 2020')
```

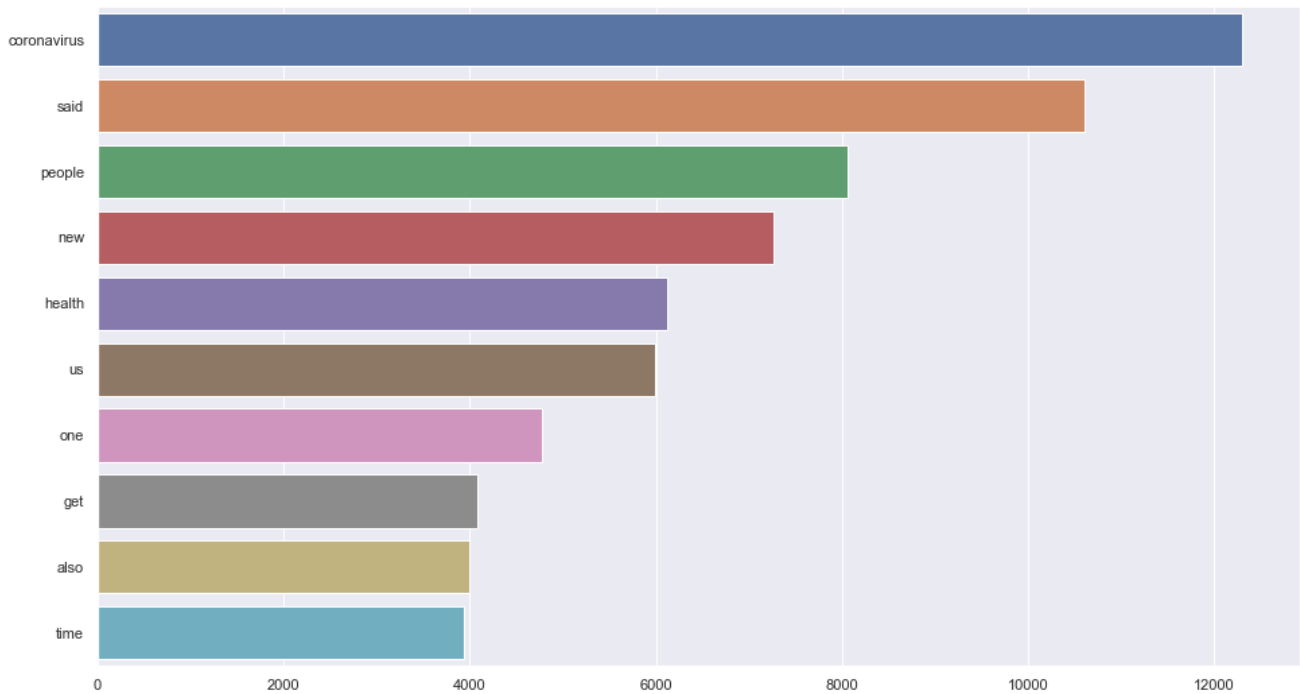
Out[72]: Text(0.5, 0.98, 'Trigram analysis of sports and entertainment news 2020')



Unigram, bigram and trigram analysis of health

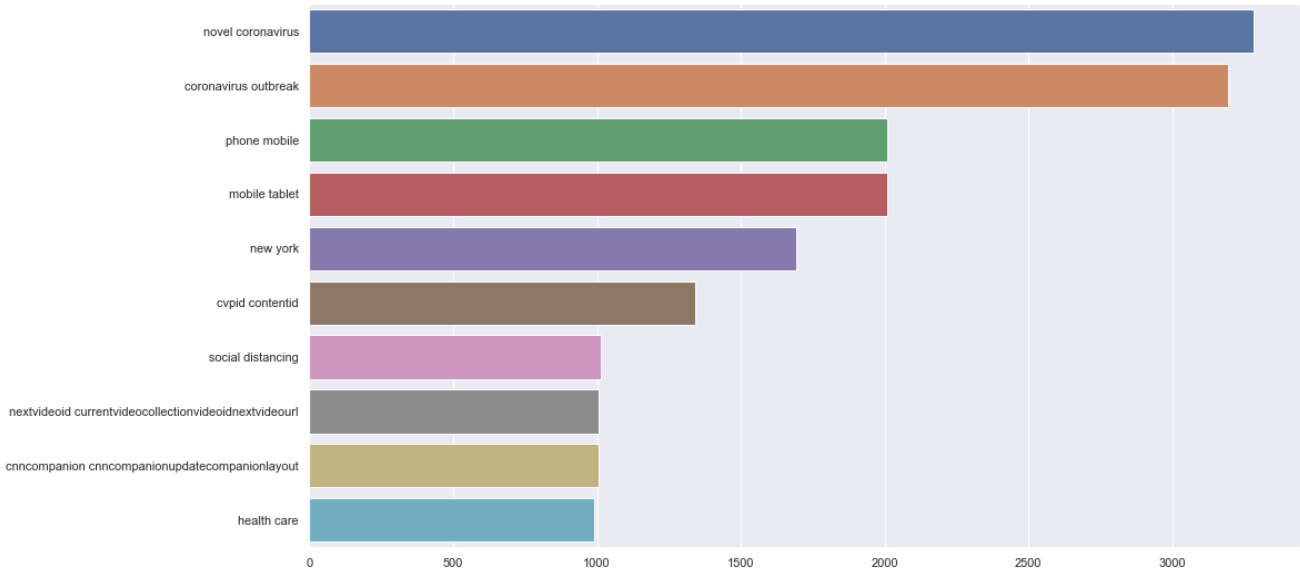
```
In [64]: plt.figure(figsize = (16,9))
most_common_uni = get_top_text_ngrams(dfahealth.text,10,1)
most_common_uni = dict(most_common_uni)
sns.barplot(x=list(most_common_uni.values()),y=list(most_common_uni.keys()))
fig.suptitle('Unigram analysis of health news 2020')
```

Out[64]: Text(0.5, 0.98, 'Unigram analysis of health news 2020')



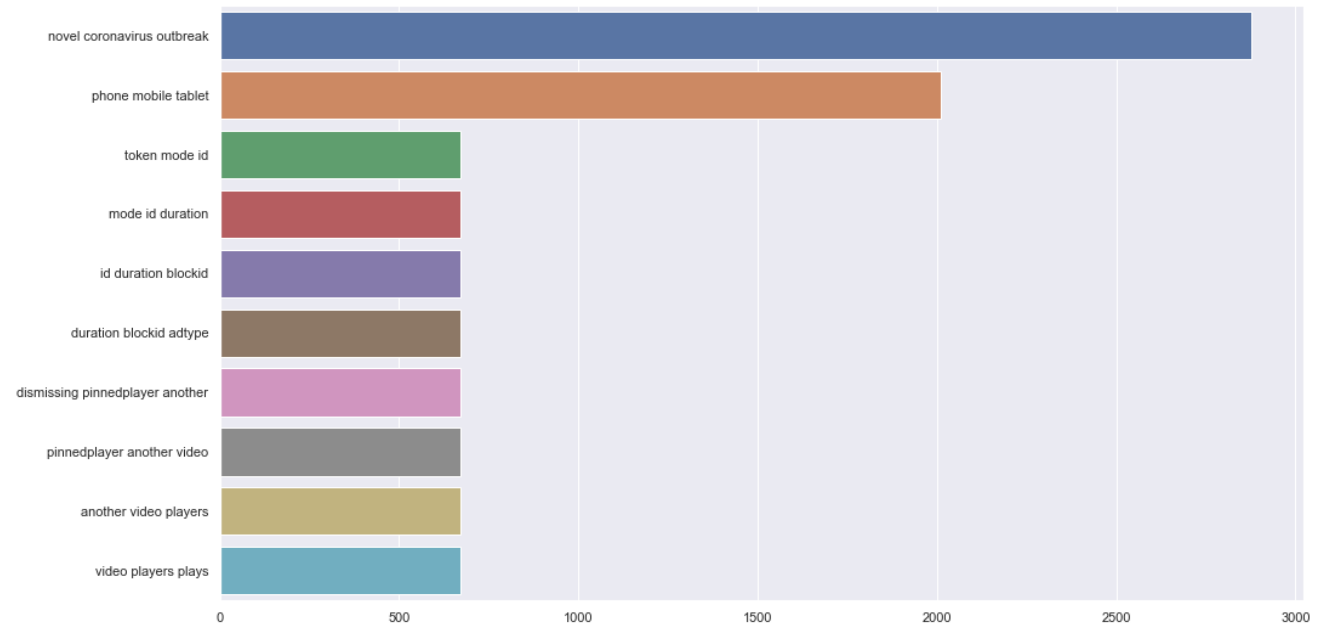
```
In [68]: plt.figure(figsize = (16,9))
most_common_bi = get_top_text_ngrams(dfahealth.text,10,2)
most_common_bi = dict(most_common_bi)
sns.barplot(x=list(most_common_bi.values()),y=list(most_common_bi.keys()))
fig.suptitle('Bigram analysis of health news 2020')
```

Out[68]: Text(0.5, 0.98, 'Bigram analysis of health news 2020')



```
In [71]: plt.figure(figsize = (16,9))
most_common_tri = get_top_text_ngrams(dfahealth.text,10,3)
most_common_tri = dict(most_common_tri)
sns.barplot(x=list(most_common_tri.values()),y=list(most_common_tri.keys()))
fig.suptitle('Trigram analysis of health news 2020')
```

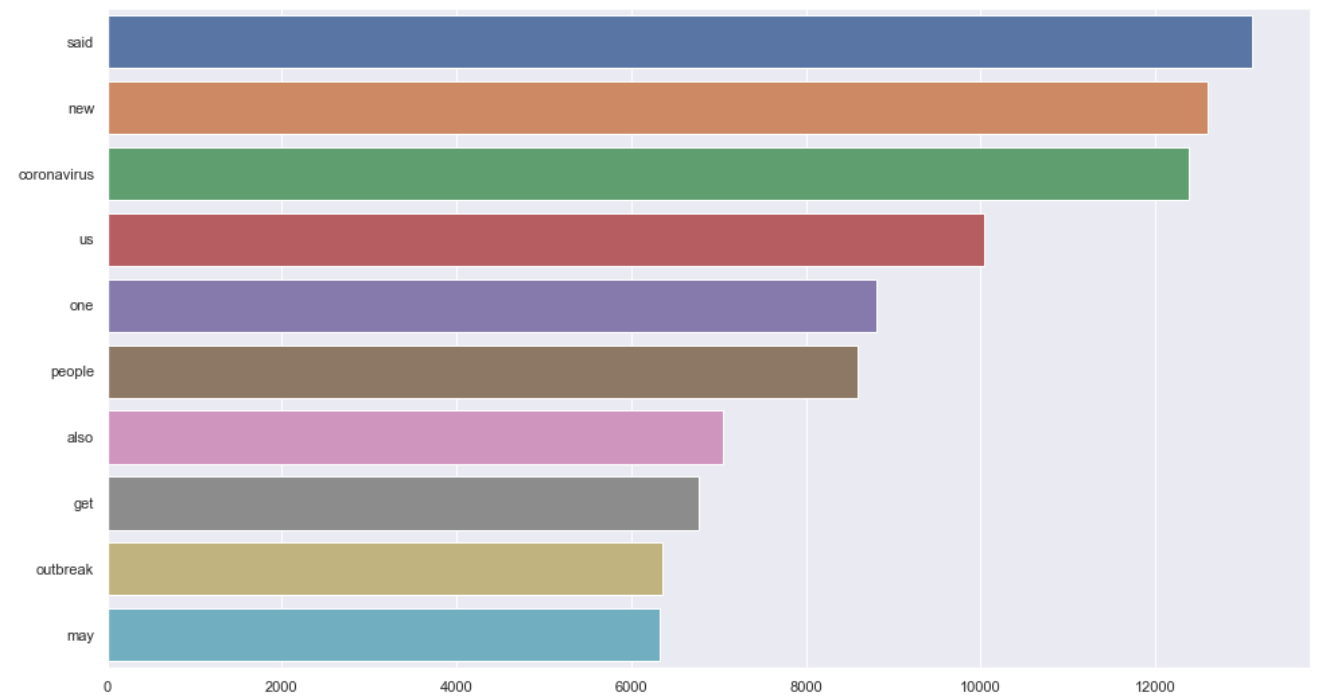
Out[71]: Text(0.5, 0.98, 'Trigram analysis of health news 2020')



Unigram, bigram and trigram analysis of business and science

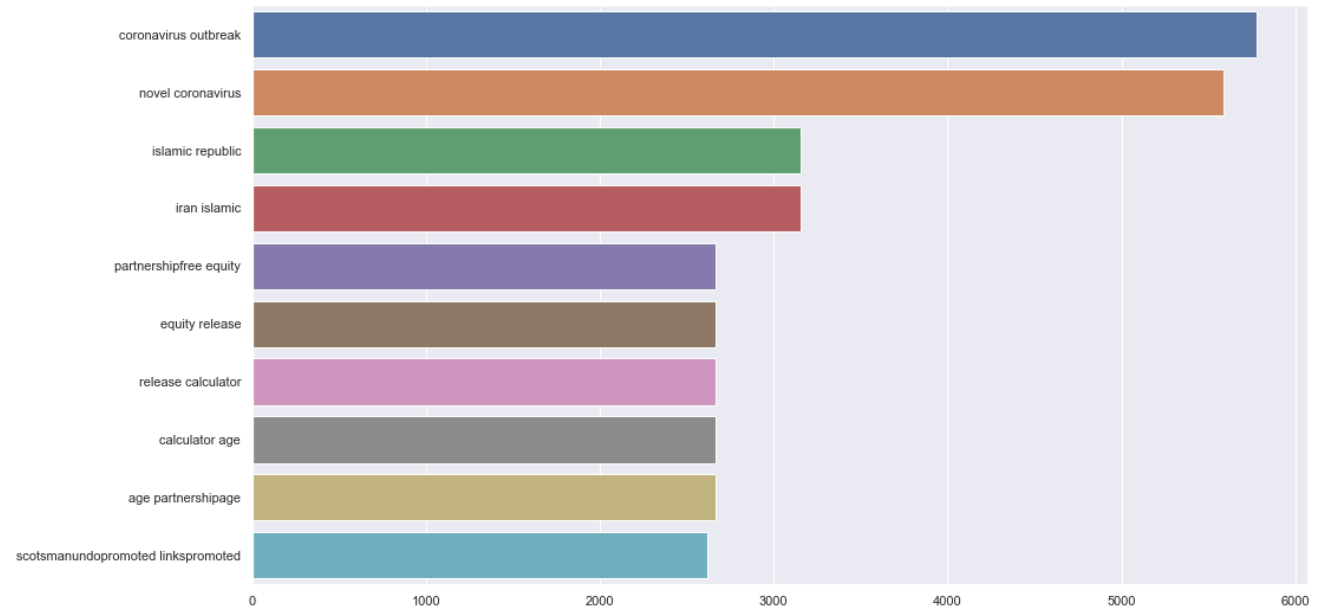
```
In [65]: plt.figure(figsize = (16,9))
most_common_uni = get_top_text_ngrams(dfabusinessandscience.text,10,1)
most_common_uni = dict(most_common_uni)
sns.barplot(x=list(most_common_uni.values()),y=list(most_common_uni.keys()))
fig.suptitle('Unigram analysis of business and science news 2020')
```

Out[65]: Text(0.5, 0.98, 'Unigram analysis of business and science news 2020')



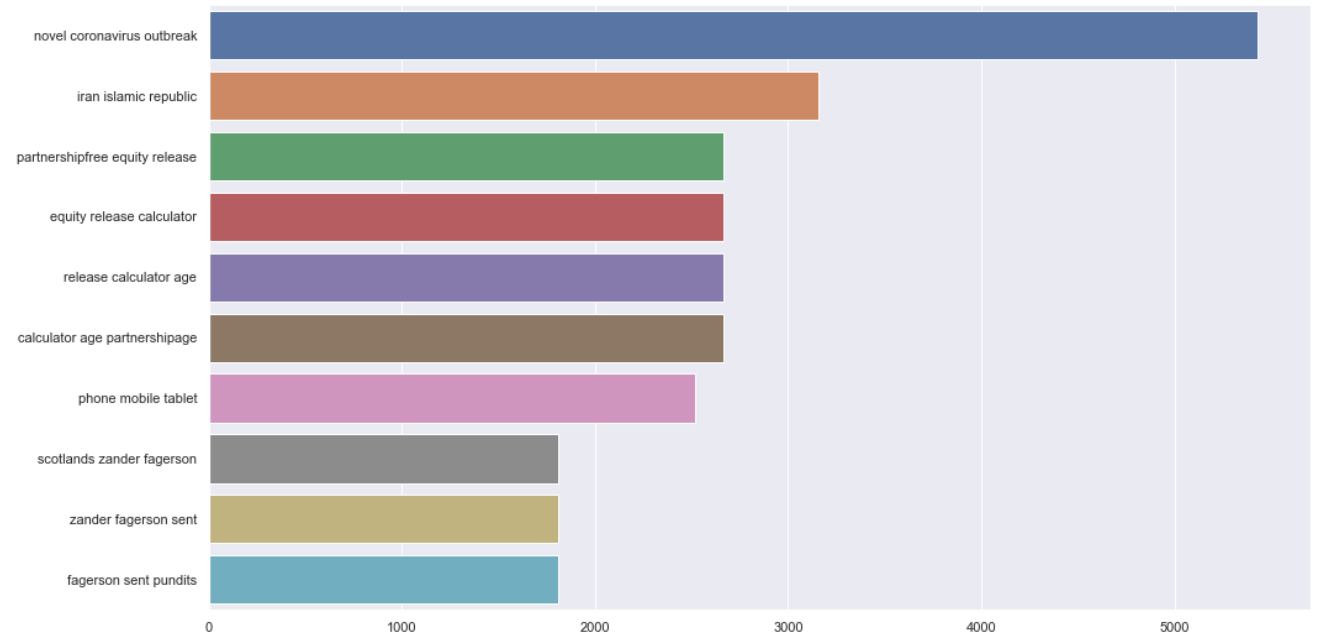
```
In [69]: plt.figure(figsize = (16,9))
most_common_bi = get_top_text_ngrams(dfabusinessandscience.text,10,2)
most_common_bi = dict(most_common_bi)
sns.barplot(x=list(most_common_bi.values()),y=list(most_common_bi.keys()))
fig.suptitle('Bigram analysis of business and science news 2020')
```

Out[69]: Text(0.5, 0.98, 'Bigram analysis of business and science news 2020')



```
In [70]: plt.figure(figsize = (16,9))
most_common_tri = get_top_text_ngrams(dfabusinessandscience.text,10,3)
most_common_tri = dict(most_common_tri)
sns.barplot(x=list(most_common_tri.values()),y=list(most_common_tri.keys()))
fig.suptitle('Trigram analysis of business and science news 2020')
```

Out[70]: Text(0.5, 0.98, 'Trigram analysis of business and science news 2020')



In []: